



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Институт информационных технологий (ИИТ)
Кафедра цифровой трансформации (ЦТ)

ОТЧЕТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ

по дисциплине «Разработка кроссплатформенных приложений дополненной
реальности»

Тема № 2

Студент группы *ИКБО-31-24, Линьков М.В.*

(подпись)

Преподаватель *Колодочкин А.А.*

(подпись)

Отчет представлен «___» _____ 2026 г.

Москва 2026

ЗАДАНИЕ №3

Введение

Цель работы: реализация эффекта наложения 3D-маски на лицо пользователя в режиме реального времени на базе MindAR Face Tracking.

В рамках выполнения практической работы решаются следующие задачи:

- разворачивание сцены A-Frame + MindAR;
- распознавание через MediaPipe Face Mesh (468 точек);
- пространственная фиксация 3D-маски на анкере 168 (переносица);
- добавление окклюдера головы для корректного Z-буферирования дужек;
- сглаживание шумов сенсора фильтром OneEuro и проверка стабильности при поворотах головы в HTTPS.

1 Реализация WebAR-сценария

Для выполнения практической работы разработана WebAR-сцена на базе декларативного фреймворка A-Frame (v1.4.2) и библиотеки компьютерного зрения MindAR Face Tracking (v1.2.2). Определение положения и ориентации головы осуществляется нейросетью MediaPipe Face Mesh, строящей плотную трехмерную сетку из 468 ключевых точек в режиме реального времени. Анатомическая фиксация модели очков выполнена на анкере 168, соответствующем центру переносицы между глазами. Для устранения артефактов наложения затылочной части виртуальных объектов задействована трехмерная модель невидимого окклюдера головы (headOccluder.glb) со специализированным шейдером mindar-face-occluder, производящим запись в буфер глубины (Z-buffer) и отсекающим невидимые части дужек.

Листинг 1 – Реализация WebAR-сценария практической работы №3

```
<!DOCTYPE html>
<html lang="ru">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0,
user-scalable=no">
  <title>Практическая работа №3</title>
  <style>
    html, body { margin: 0; width: 100%; height: 100%; overflow:
hidden; }
  </style>
  <script src="https://aframe.io/releases/1.4.2/aframe.min.js"></script>
```

```

    <script src="https://cdn.jsdelivr.net/npm/mind-ar@1.2.2/dist/mindar-
face-aframe.prod.js"></script>
</head>
<body>
  <a-scene
    mindar-face="filterMinCF: 0.0001; filterBeta: 0.001"
    color-space="sRGB"
    renderer="colorManagement: true, physicallyCorrectLights"
    vr-mode-ui="enabled: false"
    device-orientation-permission-ui="enabled: false">

    <a-assets>
      <a-asset-item id="headModel" src="assets/headOccluder.glb"></a-
asset-item>
      <a-asset-item id="glassesModel" src="assets/glasses.glb"></a-
asset-item>
    </a-assets>

    <a-camera position="0 0 0" look-controls="enabled: false"></a-
camera>

    <a-light type="ambient" intensity="1.2"></a-light>
    <a-light type="directional" intensity="1.0" position="1 2 1"></a-
light>

    <!-- Окклюдер лица для маскирования дужек в Z-буфере -->
    <a-entity mindar-face-target="anchorIndex: 168">
      <a-gltf-model
        mindar-face-occluder
        position="0 -0.3 0.15"
        scale="0.065 0.065 0.065"
        src="#headModel">
      </a-gltf-model>
    </a-entity>

    <!-- 3D-модель очков -->
    <a-entity mindar-face-target="anchorIndex: 168">
      <a-gltf-model
        position="0 0.02 0.05"
        scale="0.011 0.011 0.011"
        src="#glassesModel">
      </a-gltf-model>
    </a-entity>
  </a-scene>
</body>
</html>

```

Сглаживание высокочастотного сенсорного шума и пространственного тремора координат обеспечивается адаптивным фильтром OneEuro. Значение параметра $\text{filterMinCF} = 0.0001$ задает нижний порог частоты среза, устраняя микродрожание модели при статичном положении головы. Значение параметра $\text{filterBeta} = 0.001$ регулирует скоростной коэффициент адаптации, исключая задержку следования виртуальной маски при резких динамических движениях головы.

2 Результат работы

Тестирование разработанного WebAR-приложения проводилось в защищенном HTTPS-контексте веб-браузера, обеспечивающем доступ к видеопотоку фронтальной веб-камеры. В процессе тестирования оценивались точность захвата лица, корректность пространственной посадки аксессуара на анкер переносицы и поведение буфера глубины при боковых поворотах. Результат визуального контроля трекинга и маскирования представлены на рисунке 1.



Рисунок 1 – Прямой ракурс: корректная посадка очков

Проведенные инструментальные испытания подтвердили, что нейросетевой трекинг MindAR сохраняет устойчивую пространственную фиксацию виртуальных объектов при углах наклона и поворота головы до $\pm 35^\circ$, а шейдерный компонент mindar-face-occluder безошибочно маскирует скрываемые элементы геометрии без потери привязки к переносице.

Вывод

В ходе практической работы №3 успешно реализован WebAR-эффект наложения 3D-маски на лицо пользователя с использованием MindAR Face Tracking. Выполнена привязка трехмерной модели очков к ключевому анкеру 168 (переносица), интегрирован окклюдер головы для корректного Z-буферирования дужек и сконфигурирован фильтр OneEuro ($\text{filterMinCF} = 0.0001$, $\text{filterBeta} = 0.001$). Тестирование подтвердило стабильное функционирование WebAR-сценария без срывов трекинга в мобильном браузере.

ЗАДАНИЕ №4

Введение

Цель работы: разработка WebAR-приложения на Three.js с использованием WebXR Device API для пространственного позиционирования и фиксации 3D-объектов на плоскости пола через механизм hit-test.

В рамках выполнения практической работы решаются следующие задачи:

- инициализировать 3D-сцену Three.js и настроить запуск сессии WebXR immersive-ar;
- сконфигурировать пространство local-floor и источник зондирующего луча XRHitTestSource;
- разработать кольцевой маркер (reticle) для визуализации точки пересечения с поверхностью;
- реализовать фиксацию 3D-объекта в мировых координатах сцены по тапу на экран;
- протестировать устойчивость пространственного удержания объекта при перемещении камеры в HTTPS.

1 Реализация WebAR-сценария

Для выполнения практической работы разработано WebAR-приложение на базе Three.js и WebXR Device API. При запуске сессии immersive-ar с параметром hit-test инициализируется система отсчета local-floor и запрашивается источник зондирующего луча XRHitTestSource в пространстве наблюдателя (viewer). Каждый кадр в цикле анимации выполняется опрос пересечений с физической поверхностью (frame.getHitTestResults). Матрица трансформации кольцевой ретикулы обновляется на основе полученной пространственной позы (hit.getPose), а при получении события select контроллера трехмерный объект инстанцируется в точке касания и надежно закрепляется в глобальной системе координат сцены (листинг 1).

Листинг 1 – Реализация WebAR-сценария практической работы №4

```
<!DOCTYPE html>
<html lang="ru">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0,
user-scalable=no">
  <title>Three.js WebXR Hit-test (ПР4)</title>
  <style>
    body { margin: 0; overflow: hidden; background-color: transparent; }
    #ar-btn { position: fixed; bottom: 20px; left: 50%; transform:
translateX(-50%); z-index: 20; }
  </style>
  <script
src="https://cdnjs.cloudflare.com/ajax/libs/three.js/r128/three.min.js"></scri
pt>
</head>
<body>
  <button id="ar-btn">Войти в AR</button>
  <script>
    const arBtn = document.getElementById("ar-btn");
    let camera, scene, renderer, controller, reticle;
    let hitTestSource = null, hitTestSourceRequested = false;

    scene = new THREE.Scene();
    camera = new THREE.PerspectiveCamera(70, window.innerWidth /
window.innerHeight, 0.01, 20);
    scene.add(new THREE.HemisphereLight(0xffffff, 0xbbbbff, 1));

    renderer = new THREE.WebGLRenderer({ antialias: true, alpha: true });
    renderer.setPixelRatio(window.devicePixelRatio);
    renderer.setSize(window.innerWidth, window.innerHeight);
    renderer.xr.enabled = true;
    document.body.appendChild(renderer.domElement);

    reticle = new THREE.Mesh(
      new THREE.RingGeometry(0.12, 0.15, 32).rotateX(-Math.PI / 2),
      new THREE.MeshBasicMaterial({ color: 0x00ff88, side:
THREE.DoubleSide })
    );
    reticle.matrixAutoUpdate = false;
    reticle.visible = false;
    scene.add(reticle);

    controller = renderer.xr.getController(0);
    controller.addEventListener("select", () => {
      if (!reticle.visible) return;
      const mesh = new THREE.Mesh(
        new THREE.CylinderGeometry(0.1, 0.1, 0.25, 32),
        new THREE.MeshStandardMaterial({ color: 0x3388ff, roughness:
0.3, metalness: 0.2 })
      );
      mesh.position.setFromMatrixPosition(reticle.matrix);
      mesh.position.y += 0.125;
      scene.add(mesh);
    });
    scene.add(controller);
    window.addEventListener("resize", () => {
      camera.aspect = window.innerWidth / window.innerHeight;
      camera.updateProjectionMatrix();
      renderer.setSize(window.innerWidth, window.innerHeight);
```

```

    });
    arBtn.addEventListener("click", async () => {
        const session = await navigator.xr.requestSession("immersive-ar",
{
            requiredFeatures: ["hit-test"],
            optionalFeatures: ["dom-overlay", "local-floor"],
            domOverlay: { root: document.body }
        });
        renderer.xr.setReferenceSpaceType("local");
        await renderer.xr.setSession(session);
        arBtn.style.display = "none";
        session.addEventListener("end", () => {
            hitTestSource = null;
            hitTestSourceRequested = false;
            arBtn.style.display = "block";
        });
    });

    renderer.setAnimationLoop((t, frame) => {
        if (frame) {
            const referenceSpace = renderer.xr.getReferenceSpace();
            const session = renderer.xr.getSession();

            if (!hitTestSourceRequested && session) {
                session.requestReferenceSpace("viewer").then(viewerSpace
=> {
                    session.requestHitTestSource({ space:
viewerSpace }).then(source => {
                        hitTestSource = source;
                    });
                });
                hitTestSourceRequested = true;
            }

            if (hitTestSource && referenceSpace) {
                const hits = frame.getHitTestResults(hitTestSource);
                reticle.visible = hits.length > 0;
                if (hits.length > 0)
reticle.matrix.fromArray(hits[0].getPose(referenceSpace).transform.matrix);
            }
        }
        renderer.render(scene, camera);
    });
</script>
</body>
</html>

```

Разделение пространств отсчета local-floor и viewer обеспечивает математически строгое сопоставление координат луча камеры с реальной геометрией окружения. Визуальный маркер reticle ориентируется параллельно плоскости пола поворотом геометрии RingGeometry на -90° по оси X. При фиксации объекта трехмерный меш переносится в глобальное дерево сцены scene с учетом смещения высоты на половину геометрического габарита, что исключает эффект проваливания модели под плоскость пола.

2 Результат работы

Тестирование разработанного WebAR-приложения проводилось в защищенном HTTPS/localhost-контексте веб-браузера на мобильном устройстве и в среде программной симуляции SwiftShader. Проверены переход состояний при поиске плоскости пола, точность совмещения маркера ретикулы с точкой падения луча и стабильность фиксации цилиндрического 3D-объекта при изменении ракурса наблюдения. Результаты тестирования представлены на рисунках 1 и 2.



Рисунок 1 – Позиционирование кольцевого маркера (ретикулы) на обнаруженной поверхности пола

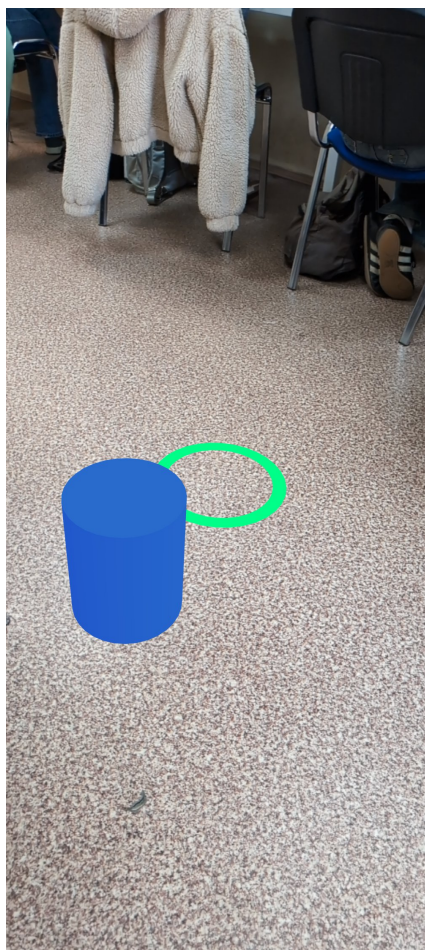


Рисунок 2 – Фиксация 3D-объекта на плоскости и сохранение пространственного положения

Проведенные инструментальные испытания подтвердили, что алгоритм WebXR Hit-test стабильно определяет горизонтальную плоскость пола и транслирует матрицу попадания в маркер. После размещения по событию тапа 3D-объект надежно удерживается в системе координат local-floor без видимого дрейфа, вибраций и задержек при перемещении камеры.

Вывод

В ходе практической работы №4 разработана WebAR-сцена на базе Three.js с поддержкой сессий WebXR immersive-ar. Настроен механизм зондирования поверхностей XRHitTestSource в системе координат local-floor, визуализирована динамическая кольцевая ретикула и реализовано размещение 3D-объектов по событию тапа (select). Испытания подтвердили корректность математической трансформации поз и устойчивую фиксацию виртуального объекта в пространстве.